

Online Chat 新手开发文档

目标读者: 新加入项目的开发 / 运维 / 测试 项目: `online-chat` (Spring Cloud 微服务 + Vue 3 SPA) 仓库: <https://github.com/liugl951127/chatInfo> 最后更新: 2026-07-08 · commit 29eb72c

0. 一句话速览

这是一个在线客服系统，支持客户↔坐席实时聊天。技术栈: - 后端: Spring Boot 3.2 + Spring Cloud Gateway + MyBatis Plus + Spring WebSocket (STOMP) + Redis - 前端: Vue 3 + Vite + Pinia + Element Plus + Axios + @stomp/stompjs v7 - 存储: MySQL 8 / MariaDB 10 + Redis 7 - 录制: html2canvas + MediaRecorder (WebM) + 分片上传 (合规录像)

5 个服务 + 1 个前端 SPA:

```
cs-auth:9001      JWT █████
cs-im:9002        ███/███/STOMP/███ (█████, █████)
cs-gateway:9000   ███ (███ + JWT █████)
cs-common         ███: ███/JWT/StompPrincipal/DTO
███:5173          Vite dev (███ 9000)
```

1. 5 分钟启动本地环境

1.1 必备依赖

工具	版本	用途
JDK	17+	后端编译运行
Maven	3.8+	后端构建
Node	18+	前端构建
MariaDB / MySQL	10.5+ / 8+	主存储
Redis	7+	在线状态 + WS 跨实例 pub/sub

1.2 一键脚本 (Debian/Ubuntu)

```
# █ JDK + Maven
apt-get update
apt-get install -y openjdk-17-jdk-headless maven

# █ Redis + MariaDB
apt-get install -y redis-server mariadb-server

# █ Redis
redis-server --daemonize yes --bind 127.0.0.1 --port 6379 --save ""

# █ MariaDB (█████ systemd █████)
mkdir -p /run/mysqld && chown mysql:mysql /run/mysqld
mariadb --user=mysql --datadir=/var/lib/mysql &
```

```
# ■■■■, ■■■■
mariadb -u root # ■■■■■■

# ■ mariadb> ■■■■ schema (9 ■■ + ■■■■■)
mariadb -u root < sql/schema.sql
```

1.3 启动后端 (4 个终端 / 4 个 nohup)

```
cd online-chat/backend

# ■ install (cs-common ■ lib, ■■■■■)
mvn -B install -DskipTests

# ■■■■■ (cs-auth / cs-im / cs-gateway ■■■■ cs-common jar, ■ install ■■■■■)
cd cs-auth      && nohup java -jar target/cs-auth.jar      > /tmp/auth.log 2>&1 &
cd ../cs-im     && nohup java -jar target/cs-im.jar        > /tmp/im.log   2>&1 &
cd ../cs-gateway && nohup java -jar target/cs-gateway.jar  > /tmp/gw.log   2>&1 &

# ■■■■
for f in auth im gw; do
  tail -f /tmp/$f.log | grep -m1 "Started Cs"
done
```

启动成功的标志: -Started CsAuthApplication -Started CsImApplication -Started CsGatewayApplication

冷启动耗时: cs-auth ≈ 15s, cs-im ≈ 12s, cs-gateway ≈ 8s (cs-im 因 WebSocket + Redis listener 较慢)

1.4 启动前端

```
cd online-chat/frontend
npm install
npm run dev -- --host 0.0.0.0 # ■■■■■■, ■■■■■■ http://<your-ip>:5173
```

1.5 验证

```
# 1. ■■
curl -s http://localhost:9000/auth/login -X POST -H "Content-Type: application/json" \
-d '{"username":"customer1","password":"123456"}' | python3 -m json.tool

# 2. ■■■■■■ http://localhost:5173
# ■ ■ customer1 / 123456 ■■ (■ agent1 / 123456)
```

2. 仓库结构

```
online-chat/
├── backend/                               ← Spring Boot ■■■■
│   ├── pom.xml                           ← ■ POM, ■■■■■■
│   ├── cs-common/                        ← ■■ jar (■■■ 3 ■■■■■)
│   │   ├── src/main/java/com/chat/common/
│   │   │   ├── constant/                ← CommonConstants (■■/■■/■■■■/Redis key)
│   │   │   ├── security/                ← JwtUtil / UserContext (ThreadLocal)
│   │   │   ├── dto/                     ← MessageDTO / ApiResponse
│   │   │   └── config/                  ← RedisConfig / JwtAuthInterceptor / WebMvcConfig
│   ├── cs-auth/                          ← ■■■■
│   ├── cs-im/                            ← ■■■■ (■■/■■/STOMP/■■)
│   └── src/main/java/com/chat/im/
```


UserContext 是 cs-common 里的 ThreadLocal, 由 JwtAuthInterceptor.preHandle 注入。

3.1.3 Redis key 命名

全部走 CommonConstants:

```
CommonConstants.REDIS_AGENT_ONLINE      = "chat:agent:online"           // Set
CommonConstants.REDIS_AGENT_SESSION     = "chat:agent:session:"         // String
CommonConstants.REDIS_CUSTOMER_SESSION  = "chat:customer:session:"       // String
CommonConstants.REDIS_WS_PUSH_CHANNEL   = "chat:ws:push"                 // Pub/Sub ■■
CommonConstants.REDIS_AGENT_STATUS      = "chat:agent:status:"           // String + TTL
// ■■: chat:user:online:{id}          (PresenceService ■)
```

新加 Redis key 一律用前缀常量, 不要在 service 里写裸字符串。

3.1.4 WebSocket 推送

单实例推:

```
messagingTemplate.convertAndSendToUser(String.valueOf(uid), "/queue/messages", dto);
```

多实例推 (cs-im 横向扩展时):

```
wsPushService.pushToUser(uid, payload); // ■ Redis pub/sub, ■■■ listener ■■■■ convertAndSendToUser
```

不要直接 messagingTemplate.convertAndSendToUser 做跨实例 — 单实例有 simpUser, 跨实例不会收到。

3.1.5 实体命名

@TableName 必须跟 SQL 一致 (默认 snake_case 由 MyBatis-Plus 命名策略自动转换)。

3.1.6 时间字段

全部用 LocalDateTime, 数据库列 DATETIME(3) 毫秒精度。不要用 Date / Instant / String。

3.1.7 审计日志

所有关键操作必须写审计:

```
auditLogService.log(uid, "ACTION_NAME", "targetId", "details");
```

现有 action 列表: LOGIN / REGISTER / CREATE_SESSION / CLAIM / TRANSFER / CLOSE / RATE / RECALL / CUSTOMER_EXIT / CUSTOMER_TRANSFER / RECORD_INIT / RECORD_END / RECORD_DENY_NO_CONSENT / RECORD_FORBIDDEN。

新增 action 在文档里登记一下, 别自己造一套。

3.2 前端

3.2.1 不要再解包一层

axios 拦截器已经解包 {code,message,data} → data:

```
// ■ ■■
const data = await imApi.mySessions() // data ■■■■■■
// ■ ■■ (■■■■ bug ■■■■)
const r = await imApi.mySessions()
if (r.code !== 0) {...} // r.code ■ undefined
console.log(r.data.sessions) // r.data ■■ undefined, ■■
```

只有裸 **fetch** 才需要手动处理 (比如 `record-sdk.js` 内部走 `fetch`)。

3.2.2 STOMP 订阅

幂等: 同一个 (destination, callback) 只订阅一次, 重复调用会被忽略。

```
stomp.subscribe('/user/queue/messages', onIncomingMessage) // ■■■: ■■■
stomp.subscribe('/user/queue/messages', onIncomingMessage) // ■■■: ■■■■ null
```

原因: 防止重连 + UI 重复订阅导致一条消息收到 2 次 (这个 bug 在 v2.0.0 修过)。

3.2.3 用户信息

永远从 **useUserStore()** 拿, 不要从 `localStorage` 直接读:

```
import { useUserStore } from '@stores/user'
const userStore = useUserStore()
const token = userStore.token
const uid = userStore.id
const nickname = userStore.nickname
```

3.2.4 路径前缀

所有 API 必须走 `/api/...` 或 `/auth/...`:

```
// ■ ■■ (Vite ■■■■■■)
http.post('/api/im/session/create', ...)
http.post('/auth/login', ...)
// ■ ■■ (Vite ■■■■■, 404)
http.post('/im/session/create', ...)
```

3.2.5 移动端适配

所有页面写完必须测手机: DevTools → Device toolbar → iPhone 12 Pro。重点: - 顶栏 fixed, 底部输入框 fixed, 中间消息流 scroll - `safe-area-inset-bottom padding` (iPhone 刘海) - 按钮 ≥ 44px 触摸目标 - 不依赖 hover 状态

4. 常见开发任务

4.1 加一个新的 REST 接口

后端 3 步:

```
// 1) Controller (cs-im/src/main/java/com/chat/im/controller/XxxController.java)
@PostMapping("/foo")
public ApiResponse<MyDto> foo(@RequestBody MyDto in) {
    Long uid = UserContext.userId();
    return xxxService.doFoo(uid, in);
}
```

```

}

// 2) Service
@Service
@RequiredArgsConstructor
public class XxxService {
    @Transactional
    public ApiResponse<MyDto> doFoo(Long uid, MyDto in) {
        // ■■
        auditLogService.log(uid, "F00", null, "details");
        return ApiResponse.ok(result);
    }
}

// 3) ■■ API
// frontend/src/api/im.js (■■■■■■■■ .js)
foo: (data) => http.post('/api/im/foo', data),

```

4.2 加一个新的 WebSocket 事件类型

后端:

```

// WsPushService.java
public void notifyFoo(Long peerId, SomeDto payload) {
    Map<String, Object> ev = new HashMap<>();
    ev.put("type", "F00");
    ev.put("data", payload);
    // ■■■: ■■■; ■■■: ■ pushToUser ■■■
    pushToUser(peerId, ev);
}

```

前端:

```

// Agent.vue / Customer.vue onEvent
if (payload.type === 'F00') {
    // ■■
}

```

注意: `WsPushService.onMessage` 现在能正确处理 Map (不再强行转 `MessageDTO`), 见 `parse()` 方法。

4.3 加一张新表

1. SQL: 在 `sql/schema.sql` 加 `CREATE TABLE`, 字段顺序和 entity 字段一致
2. Java entity: `@TableName("xxx") @Data public class Xxx { ... }`
3. Mapper: `extends BaseMapper<Xxx>`, 加到 MyBatis-Plus 自动扫描路径
4. (可选) 加到 `chat_audit_log` 关联: 该表已自动按 `created_at` 索引

4.4 录制 SDK 加新配置项

`frontend/src/utils/record-sdk.js` 的 `opts` 直接加新字段, 默认值在构造函数 `merge`:

```

this.opts = {
    ...opts,
    myNewOption: opts.myNewOption ?? 'default',
}

```

同时在 ChatRecordSDK 类的 JSDoc 注释里更新参数说明。

4.5 改前端后强制刷新

Vite 默认 HMR, 但有些改动 (router / main.js) 需要手动刷新。生产构建 (Vite 用 hash 文件名) 可以强制刷新清缓存:

```
Vite --force (■ .vite ■■)
■■■■: DevTools → Network → Disable cache
```

5. 调试技巧

5.1 后端日志级别

application.yml 默认 com.chat: DEBUG。想看更细:

```
logging:
  level:
    com.chat.im: DEBUG
    org.springframework.web.socket: INFO
    org.springframework.messaging: INFO
```

5.2 STOMP 调试

打开 ws-client.js 里的 debug: () => {} 改成 console.log, 可以看到所有 STOMP 帧。

5.3 录制 SDK 调试

浏览器 DevTools: 1. Network 面板: 看 /api/im/record/chunk 上传 (multipart/form-data) 2. Console 面板: SDK 打 [record] 前缀的日志 3. Application → Service Workers: 看注册情况 (SDK 不需要 SW) 4. Memory 面板: 长时间录制注意 chunkQueue 是否泄漏

5.4 数据库调试

```
mariadb -u root online_chat
```

常用查询:

```
-- ■■■■■■
SELECT id, session_no, customer_id, agent_id, status, created_at
FROM chat_session ORDER BY id DESC LIMIT 10;

-- ■■■■■■
SELECT id, session_id, user_id, end_reason, chunk_count, total_bytes, consent_given
FROM chat_record ORDER BY id DESC LIMIT 10;

-- ■■■■■■
SELECT action, target, detail, created_at
FROM audit_log ORDER BY id DESC LIMIT 20;

-- ■■■■■■■■ (■■■■■)
SELECT actor_id, action, target, detail, ip, created_at
FROM chat_audit_log
WHERE action LIKE 'RECORD_%' ORDER BY id DESC LIMIT 20;
```

5.5 Redis 调试

```
redis-cli
```

常用:

```
KEYS chat:agent:online          # 4
KEYS chat:user:online:*         # 4
HGETALL chat:agent:status:4     # 4
LRANGE chat:offline:msg:2 0 -1 # 2
PUBSUB CHANNELS 'chat:ws:push:*' # WS
```

5.6 WebSocket 抓包

wscat 或浏览器 DevTools → Network → WS 面板, 可以看到所有 STOMP 帧。

```
# (Node)
npx wscat -c 'ws://localhost:9000/ws/customer?token=xxx'
> CONNECT
accept-version:1.2
```

6. 部署清单

6.1 生产部署要点

1. 用 nginx 替代 Vite, 配置:

```
server {
    listen 80;
    server_name chat.example.com;

    # SPA
    root /var/www/online-chat/dist;
    index index.html;

    # SPA history mode fallback
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API
    location /auth/ { proxy_pass http://127.0.0.1:9000; }
    location /api/ { proxy_pass http://127.0.0.1:9000; }

    # WebSocket (Upgrade)
    location /ws/ {
        proxy_pass http://127.0.0.1:9000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 86400; #
    }
}
```

1. JWT secret 用环境变量:

```
export CHAT_JWT_SECRET="$(openssl rand -hex 32)"
```

三个服务 (cs-auth / cs-im / cs-gateway) 必须用同一个, 否则 token 解析失败。

1. 录制存储:
2. 开发: /tmp/chat-records (临时)

生产: 对象存储 (S3/MinIO) + 数据库只存路径 + 后台 cron 清理 30 天前

日志: 接入 ELK / Loki, 重点监控:

5. chat_audit_log.action IN ('RECORD_DENY_NO_CONSENT', 'RECORD_FORBIDDEN') (合规异常)
6. cs-im 启动失败 / OOM
7. Redis 连接失败

6.2 横向扩展

cs-im 可起多个实例, 需要: - Nginx 用 ip_hash (或 sticky cookie) 路由 /ws/** 保持会话粘性 - Redis Pub/Sub 已支持跨实例 WS 推送 - 数据库共享 MySQL + Redis - 注意: localRegistered 是本实例的, 跨实例 PRESENCE 事件通过 Redis pub/sub 触发 @EventListener, 行为正确

cs-auth / cs-gateway 完全无状态, 直接 LB。

6.3 监控指标

- /actuator/health (Spring Boot 默认)
- WebSocket 连接数: cs-im 日志 BrokerAvailabilityEvent
- Redis 内存: INFO memory
- 录制存储: du -sh /var/lib/chat-records/

7. 故障排查 (FAQ)

7.1 "登录 401"

- 检查 token 是否过期 (24h)
- 检查三个服务 JWT secret 是否一致 (cs-auth / cs-im / cs-gateway 的 chat.jwt.secret)
- 检查 UserContext 是否正确注入 (grep "UserContext.userId" backend/cs-im/...)

7.2 "WS 连不上"

- 检查 HandshakeAuthInterceptor 是否正确提取 token
- 浏览器用 ?token=xxx (WS 不能自定义 header)
- 工具/Postman 用 Authorization: Bearer xxx header
- nginx 部署时检查 Upgrade / Connection 头是否透传

7.3 "消息双发 / 漏发"

- 双发: 检查 subscribe() 是否幂等, 不能在 onConnected 里再订阅一次

- 漏发: 检查 `wsPushService.parse()` 是否能解析 payload (Map vs MessageDTO)
- 离线: 客户断网时 `presenceService.isOnline=false` → 消息落 Redis List, 客户重连后 `drainOffline()` 拉

7.4 "录制 404 / 上传失败"

- 检查 `/api/im/record/*` 路由 (Vite 代理要配 `/api` 前缀)
- 检查 `multipart.max-file-size` 配置 (默认 50MB, 视频可能更大)
- 检查 `record.storage-path` 是否可写
- 检查 `chat_record.consent_given` 必须是 1 (前端调 init 时必须 `consent=true`)

7.5 "回放视频只有前几秒"

- WebM 直拼只能保证首段可播 (后续分段浏览器不识别)
- 解决: 服务端用 `ffmpeg remux` 或前端 MSE 流式 `appendBuffer`
- 当前是已知限制, 文档已标注

7.6 "Vite 改了不生效"

- 浏览器硬刷新 (Ctrl+Shift+R / Cmd+Shift+R)
- Vite 加 `--force` 重新启动
- 检查 `node_modules/.vite` 是否需要清

8. 安全清单

- [x] JWT HS512, secret 32+ 字节, 不硬编码 (用环境变量)
- [x] 所有 API 走 `JwtAuthInterceptor` (白名单除外)
- [x] BCrypt rounds=10 (不可逆)
- [x] WebSocket token 校验 (`HandshakeAuthInterceptor`)
- [x] 录制必须 `consentGiven=true` (后端再次校验)
- [x] 录像下载三道权限 (本人/该会话坐席/admin)
- [x] 跨域: 浏览器走 Vite/nginx 代理, 同源
- [x] 审计日志: 所有关键操作留痕 (含 IP + UA)
- [] 待办: 限流 (sentinel / gateway filter)
- [] 待办: 防 CSRF (目前 JWT 在 Authorization header, 防 CSRF 还需加 SameSite cookie 或 double-submit)

9. 性能基线 (单实例, 8C16G)

接口	并发	avg	max	备注
POST /auth/login	100	118ms	826ms	BCrypt 占大头, 缓存可优化
GET /im/session/mine	50	77ms	224ms	经 gateway

GET /im/record/{id}/chunks	50	30ms	80ms	静态文件
GET /im/record/chunk/{id}/raw	50	15ms	40ms	
POST /im/agent/link	串行	30-75ms	75ms	JIT 后

10. 路线图

- [] 录像 ffmpeg 合流 (回放无断点)
- [] 客户录像合规授权 ToS 入口 (登录时一次告知)
- [] 限流 (gateway sentinel)
- [] Spring Cache (Redis Caffeine) 缓存热数据
- [] 多语言 (i18n)
- [] 客服工作量统计 (报表)
- [] 客服在线状态查询 (REST 给前端轮询)
- [] 客户排队可视化
- [] 智能分配升级 (按客服当前会话数 / 历史满意度)

11. 相关文档

文档	内容
API_CHAIN.md	接口链路 (REST + WS + 关键流程)
DEV_GUIDE.md	本文档 (开发维护)
sql/schema.sql	数据库 DDL + 演示数据
README.md	项目总览
dist/	前端生产构建产物 (Vite build)

有问题先看 `API_CHAIN.md` 和本指南的"故障排查"。开发相关 issue 在 GitHub 提。